

Resumen Tecnología Digital VI

Segunda parte

Juan Ignacio Elosegui

1. Introducción a las redes neuronales

1.1. Del mundo tabular al no estructurado

En el **mundo tabular**, los datos son estructurados y las columnas tienen semántica específica para cada problema. Los modelos se encargan de hallar interacciones entre columnas.

En el **mundo no estructurado** (texto, imágenes, audio), existe una relación general entre los datos más allá del problema puntual. Las redes neuronales funcionan mejor aquí.

1.2. Perceptrón

Perceptrón

Red de una sola neurona. Usa una función heaviside step. Lo único que se puede modificar son los pesos $\vec{w}$.

Puede aprender AND y OR, pero **no puede aprender XOR** porque no es linealmente separable.

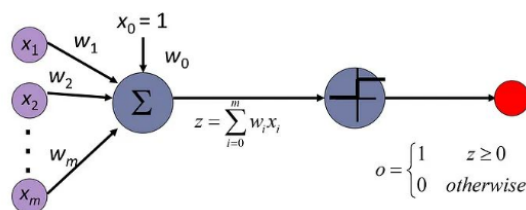


Figura 1: Esquema del perceptrón.

Los datos entran por $\vec{x}$ (input) y las respuestas salen por $\vec{y}$ (output). El **bias** $\vec{b}$ es un peso especial que no multiplica a ninguna variable.

1.3. Arquitecturas: Perceptrón Multicapa (MLP)

Varias neuronas en paralelo e interconectadas forman arquitecturas que sí resuelven problemas.

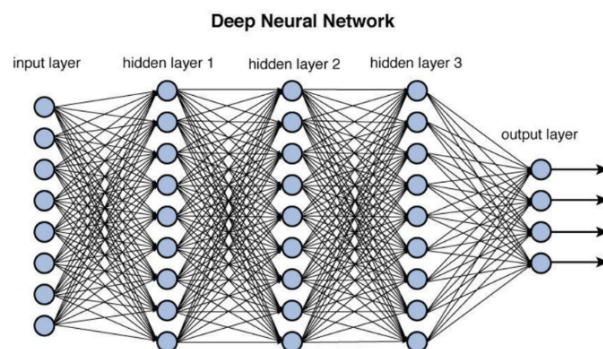


Figura 2: Red neuronal feedforward fully-connected.

- **Feedforward**: la información va hacia adelante.
- **Fully-connected**: todas las neuronas de una capa conectan con todas las de la siguiente.
- Capas: **input**, **ocultas** y **output**.
- Más capas en serie = **deep learning**.

Si no se usan funciones de activación no lineales, apilar neuronas colapsa en una sola transformación lineal. Sin romper la linealidad, no tiene sentido tener múltiples capas.

Backpropagation propaga los gradientes de la loss hacia atrás usando la regla de la cadena, indicando cuánto y en qué dirección actualizar cada peso.

Tensor: matrices multidimensionales (estructura base en PyTorch).

2. Forward y Backward Propagation

2.1. Entrenamiento

Se optimizan los parámetros (pesos y biases) para predecir correctamente el conjunto de entrenamiento:

- **Forward**: se evalúa la red y se mide el error con la loss function.
- **Backward**: se corrigen los pesos usando regla de la cadena, derivadas parciales y learning rate.

2.2. Inferencia

Solo se usa forward propagation (sin actualización de pesos).

2.3. Vectorización

La forma matricial del cómputo de una capa es:

$$z = W^T \vec{x} + \vec{b}$$

donde W es la matriz de pesos, $\vec{x}$ el vector de inputs y $\vec{b}$ el vector de biases.

La **loss function** promedia los errores sobre todas las muestras:

$$\mathcal{J}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

3. Funciones de activación

3.1. Complicaciones

3.1.1. Vanishing Gradient

Ocurre en redes profundas cuando los gradientes se vuelven muy pequeños al propagarse hacia las primeras capas. Funciones como sigmoide o tanh tienen derivada casi cero en sus extremos, y multiplicar muchos números pequeños achica aún más el resultado.

Consecuencia: las primeras capas casi no actualizan sus pesos $\Rightarrow$ no aprenden. La red no puede aprender representaciones profundas.

3.1.2. Zig-Zag

Si la función de activación tiene salidas siempre positivas (como sigmoide), todos los gradientes de los pesos de esa capa tienen el mismo signo, forzando a todos los pesos a moverse en la misma dirección.

Solución: funciones centradas en cero (como tanh) o normalizar los inputs.

3.1.3. Dying ReLU

La derivada de ReLU es exactamente cero para valores negativos, produciendo neuronas muertas que nunca actualizan sus pesos. Puede ser bueno (esparsidad $\Rightarrow$ regularización) o malo (mala inicialización $\Rightarrow$ entrenamiento inútil).

Soluciones: Leaky ReLU, Swish, GELU.

3.2. Catálogo de Funciones

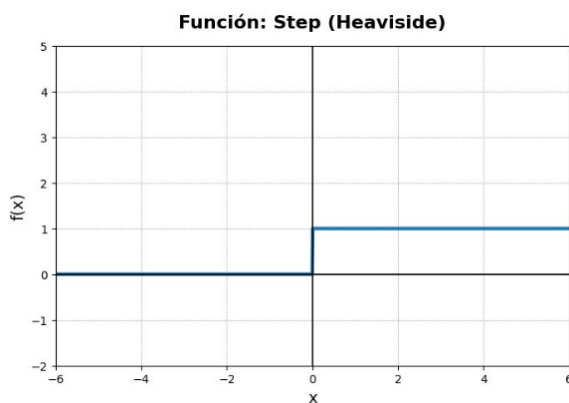


Figura 3: Heaviside Step.

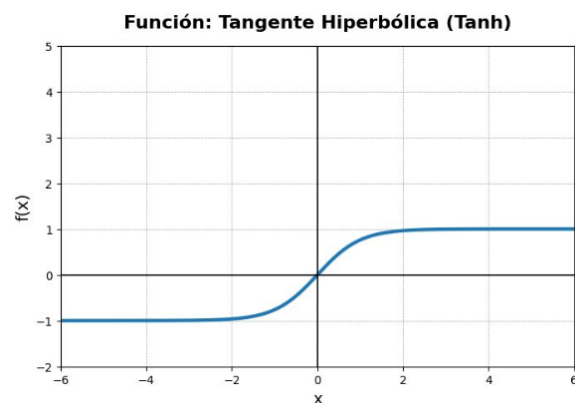


Figura 4: Tanh.

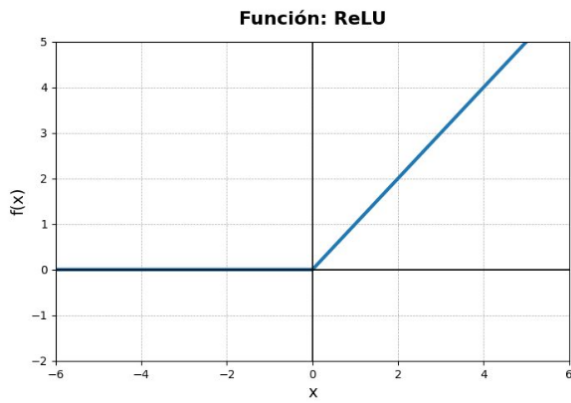


Figura 5: ReLU.

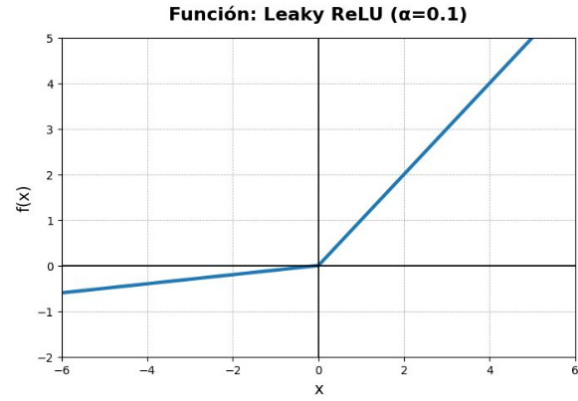


Figura 6: Leaky ReLU.

Función	Fórmula	Características
Heaviside	$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$	Derivada 0 $\Rightarrow$ no sirve para backprop
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	Centrada en 0, sufre vanishing
ReLU	$\max(0, x)$	Barata, no acotada, no centrada en 0
Leaky ReLU	$\begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$	Evita dying ReLU
Swish	$x \cdot \sigma(x)$	Suave, más costosa que ReLU
GELU	$x \cdot \Phi(x)$	Buena con Transformers

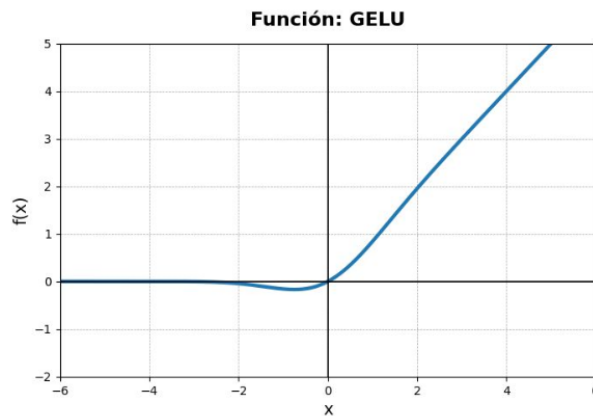


Figura 7: Comparación GELU, Swish y otras funciones.

3.3. Softmax

Toma todo el vector de salidas y lo transforma en probabilidades (entre 0 y 1). Se usa en la última capa para clasificación de clase única:

$$f(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

4. Optimización e hiperparámetros

4.1. Loss Landscape

La loss function suele tener formas irregulares y no se minimiza de forma analítica. Se usan métodos iterativos comenzando con pesos inicializados aleatoriamente.

4.2. Variantes de Gradient Descent

Gradient Descent y variantes

- **Batch GD (BGD)**: usa todo el dataset por paso. Estable pero costoso en datasets grandes.
- **Stochastic GD (SGD)**: usa una muestra al azar por paso. Barato pero ruidoso.
- **Mini-batch**: agrupa n muestras. Balance entre BGD y SGD.

Época = pasar todos los datos de entrenamiento (divididos en mini-batches) por la red.

4.3. Problemas de terreno

- **Ravine**: topología tipo cañón $\Rightarrow$ zig-zag.
- **Mesetas y puntos silla**: pendiente casi cero $\Rightarrow$ SGD no progresa.

Momentum: los gradientes ya no son independientes; se acumulan gradientes de pasos anteriores para suavizar la trayectoria.

4.4. Learning Rate adaptativo

Algoritmo	Idea
AdaGrad	LR adaptado por parámetro, inversamente proporcional a la acumulación de gradientes al cuadrado.
RMSProp	Decaimiento exponencial de la acumulación para olvidar gradientes viejos.
ADAM	Combina momentum con LR adaptativo.

4.5. Schedulers

Adaptación *temporal* del LR global por épocas:

- **Step decay**: cada n épocas se divide por 2.
- **Exponential decay**: multiplicar por un escalar < 1 constantemente.
- **Cosine Annealing**: LR sigue una curva de coseno descendente.
- **Reduce on Plateau**: si la loss no mejora en varias épocas, reducir LR a la mitad.

4.6. Hiperparámetros

Parámetros que se definen de antemano (no los aprende la red). Estrategias de búsqueda:

1. Random Search
2. Grid Search
3. Optimización Bayesiana

4.7. Input Scaling

Las redes son sensibles a los rangos de los inputs. Estandarizar allana el loss landscape y mejora la convergencia.

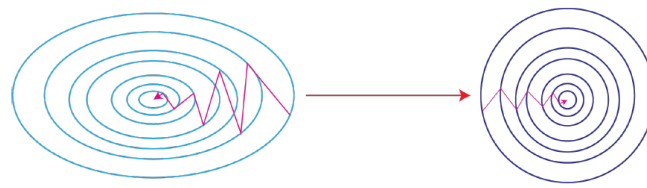


Figura 8: Efecto del input scaling en el loss landscape.

4.8. Inicialización de pesos

La convergencia depende directamente del punto de partida. Una buena inicialización evita vanishing/exploding gradients. Métodos: Random, LeCun, Xavier.

4.9. Internal Covariate Shift y Batch Normalization

Internal Covariate Shift

Aunque se controle la distribución de entrada, las capas ocultas cambian la distribución de sus salidas durante el entrenamiento, obligando a las capas siguientes a readaptarse constantemente.

Batch Normalization: normaliza los valores de cada batch antes de ciertas capas, corrigiendo este problema y actuando como regularización.

4.10. Regularización

4.10.1. Early Stopping

Cortar el entrenamiento cuando las curvas de loss de train y validación comienzan a diverger.

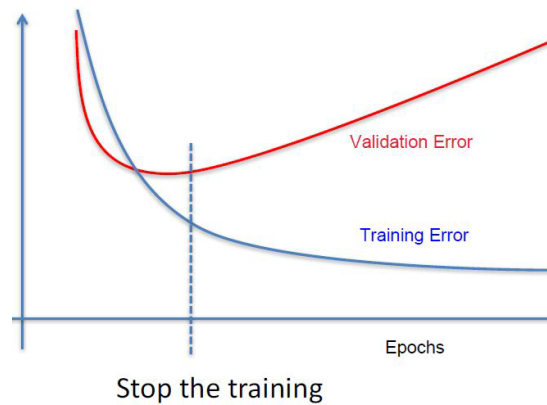


Figura 9: Early stopping: punto óptimo antes del sobreajuste.

4.10.2. L1 y L2

Funciones de presupuesto sumadas a la loss function para restringir los valores de los pesos.

4.10.3. Dropout

En cada mini-batch se apagan neuronas al azar, evitando que ciertos patrones sean memorizados. Solo afecta al entrenamiento; en inferencia se usan todas.

4.10.4. Monitoreo de Gradientes

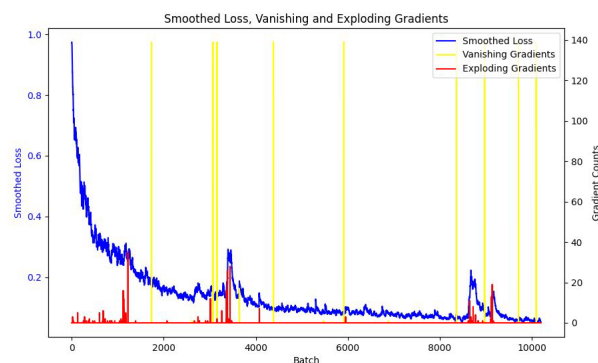


Figura 10: Monitoreo: picos en la loss $\leftrightarrow$ exploding gradients; estancamiento $\leftrightarrow$ vanishing.

5. Redes Neuronales Convolucionales (CNNs)

5.1. Motivación

En datos no tabulares (imágenes), los píxeles no son independientes entre sí. Una red fully-connected ignora la estructura espacial. Las CNNs explotan la relación entre píxeles vecinos.

Una imagen es una matriz (o tres matrices para RGB) que contiene números.

5.2. Convolución

Convolución 2D discreta

Operación que toma una imagen I y un kernel K (filtro) pequeño. El kernel se desplaza por la imagen, multiplicando elemento a elemento y sumando:

$$(I * K)[i, j] = \sum_m \sum_n I[i + m, j + n] \cdot K[m, n]$$

- **Padding:** rellenar bordes con ceros para controlar el tamaño de salida.
- **Stride:** el “tranco” que da el filtro. Stride grande $\Rightarrow$ salida más chica.

Fórmula del tamaño de salida:

$$O = \frac{I - K + 2P}{S} + 1$$

Ejemplo: imagen 28×28 , kernel 3×3 , $P = 1$, $S = 2$: $O = \frac{28-3+2}{2} + 1 = 14$. Salida: 14×14 .

Los filtros (kernels) se inicializan random y se ajustan con backpropagation. La red aprende qué detectar.

5.3. Pooling

Resume información de un grupo de píxeles vecinos:

- Achica la dimensión (ahorra espacio y cómputo).
- Aumenta el campo receptivo.
- Da robustez ante traslación.

5.4. Bloque convolucional

Bloque convolucional estándar

Convolución $\rightarrow$ Función de activación $\rightarrow$ Pooling

La convolución es una operación lineal; la no linealidad la aportan la activación y el pooling.

5.5. Ejemplo: parámetros MLP vs CNN

Para una imagen de $32 \times 32 \times 3$ (color):

MLP: flatten + capa de 128 neuronas + capa de 10 salidas = $(3072 \times 128 + 128) + (128 \times 10 + 10) = 394,634$ parámetros.

CNN: conv de 10 filtros 5×5 + maxpool 2×2 + conv de 20 filtros 3×3 = $760 + 1,820 = 2,580$ parámetros.

La CNN tiene $\sim 150\times$ menos parámetros gracias al parameter sharing y la localidad.

6. Arquitecturas famosas

Todas fueron evaluadas en el **ImageNet Challenge** (14M imágenes, 22K clases).

6.1. AlexNet (2012)

Primera CNN profunda importante para aplicaciones reales. Ganó ImageNet por mucho.

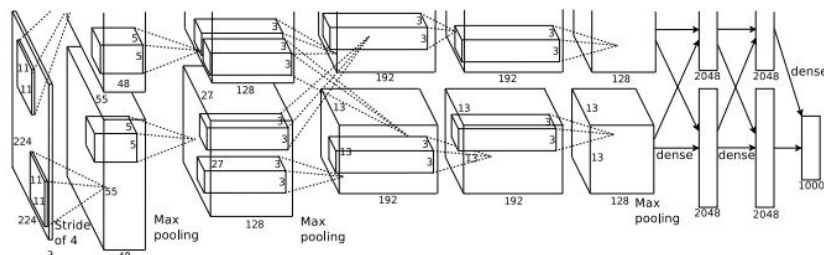


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network’s input is 150,528-dimensional, and the number of neurons in the network’s remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Figura 11: Arquitectura AlexNet.

Aportes: entrenamiento multi-GPU, capas de normalización, data augmentation, dropout, ensamble de CNNs.

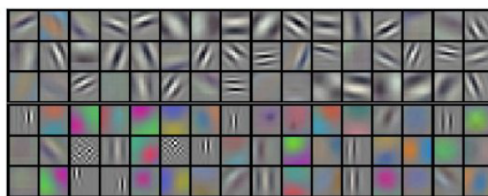


Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

Figure 3 shows the convolutional kernels learned by the network’s two data-connected layers. The network has learned a variety of frequency- and orientation-selective kernels, as well as various colored blobs. Notice the specialization exhibited by the two GPUs, a result of the restricted connectivity described in Section 3.5. The kernels on GPU 1 are largely color-agnostic, while the kernels on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization (modulo a renumbering of the GPUs).

Figura 12: Kernels aprendidos por la primera capa convolucional de AlexNet.

6.2. ZFNet (2013)

Mejoró a AlexNet con ajustes de hiperparámetros. Su contribución principal fue la interpretación visual de lo que aprenden las CNNs.

6.3. VGG (2014)

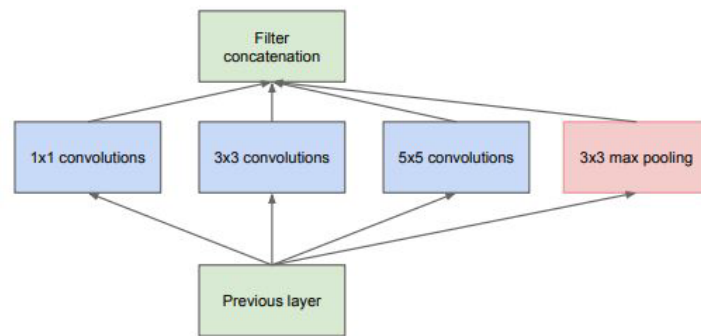
Demostró que usar siempre **filtros de 3×3 con stride 1** combinados secuencialmente obtiene el mismo campo receptivo que filtros más grandes, con más flexibilidad y menos parámetros. Red profunda y pesada. Buenas representaciones transferibles.

6.4. GoogLeNet / Inception (2014)

Inception Module

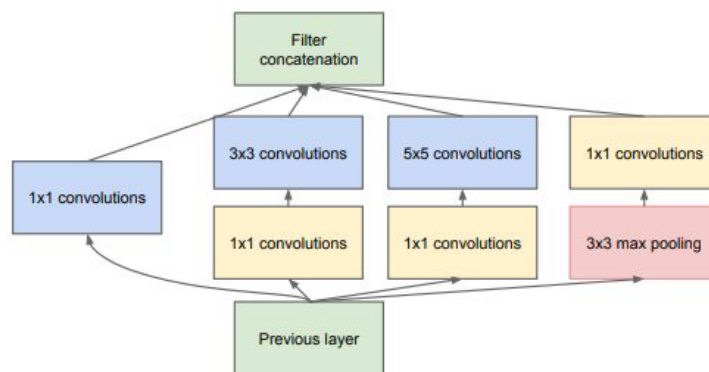
En vez de elegir un tamaño de filtro, se aplican **distintas convoluciones y max pooling en paralelo**, concatenando las salidas. La red elige el tamaño adecuado. **Convoluciones 1×1** : reducen la cantidad de canales (reducción de dimensionalidad aprendida).

Global Average Pooling: reemplaza las capas Flatten antes de la clasificación final, reduciendo drásticamente los parámetros.



(a) Inception module, naïve version

Figura 13: Inception Module básico.



(b) Inception module with dimension reductions

Figura 14: Inception Module con convoluciones 1×1 para reducción.

22 capas con solo $\sim 5\text{M}$ parámetros (vs. $\sim 140\text{M}$ de VGG).

6.5. ResNet (2015)

Bloque residual

Se agrega una **skip connection**: el input se bifurca, pasa por convoluciones por un lado y se saltea por otro, sumándose antes de la activación:

$$H(x) = F(x) + x$$

Ventajas:

1. Agregar capas nunca empeora: el bloque puede aprender $F(x) = 0$ (identidad).
2. El gradiente de la skip connection es 1 $\Rightarrow$ mitiga vanishing gradients.

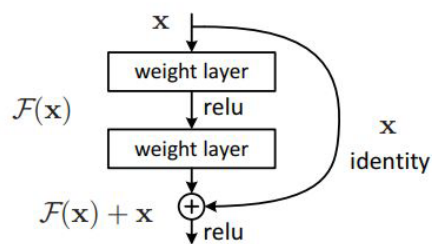


Figure 2. Residual learning: a building block.

Figura 15: Bloque residual con skip connection.

Error del 3 % en ImageNet. Terminó efectivamente la competencia.

7. Visualización e interpretabilidad de CNNs

7.1. Redes Deconvolucionales (Zeiler & Fergus)

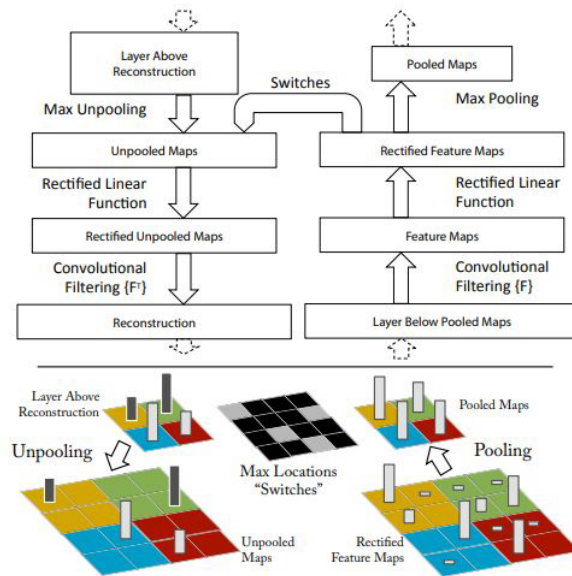


Figure 1. Top: A deconvnet layer (left) attached to a convnet layer (right). The deconvnet will reconstruct an approximate version of the convnet features from the layer beneath. Bottom: An illustration of the unpooling operation in the deconvnet, using *switches* which record the location of the local max in each pooling region (colored zones) during pooling in the convnet.

Figura 16: Visualización: primeras capas captan patrones generales; últimas capas, detalles refinados.

7.2. Oclusión

Se tapa una parte de la imagen y se observa cómo afecta la predicción.

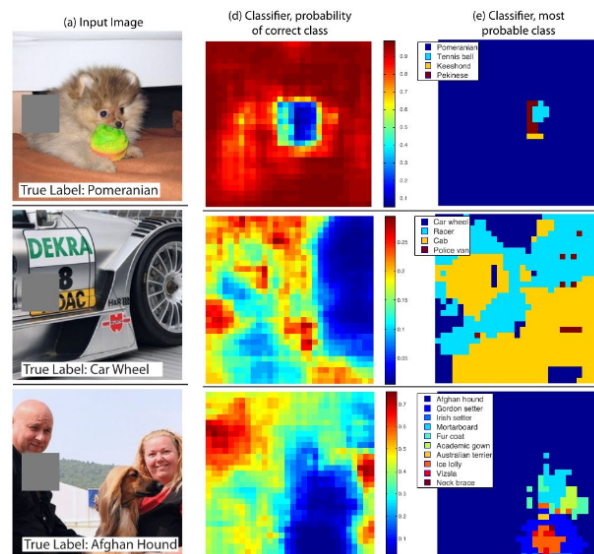


Figura 17: Sensibilidad a la oclusión: qué partes de la imagen influyen en la predicción.

7.3. Espacio latente e Image Similarity

La última capa antes de clasificar codifica imágenes en un espacio de features. Imágenes similares quedan cerca en ese espacio. Se aplican técnicas de reducción de dimensionalidad para visualizar.



Figura 18: Visualización del espacio latente con reducción de dimensionalidad.

7.4. Saliency Maps

Saliency Maps

En una única pasada hacia atrás, se calcula la sensibilidad de la predicción respecto a cada píxel. Los píxeles con mayor gradiente son los más importantes para la clasificación. Se trata al input como variable y a los pesos como fijos.

7.5. Grad-CAM

En vez de calcular píxel a píxel, calcula la importancia por parches, generando mapas de calor sobre la imagen.

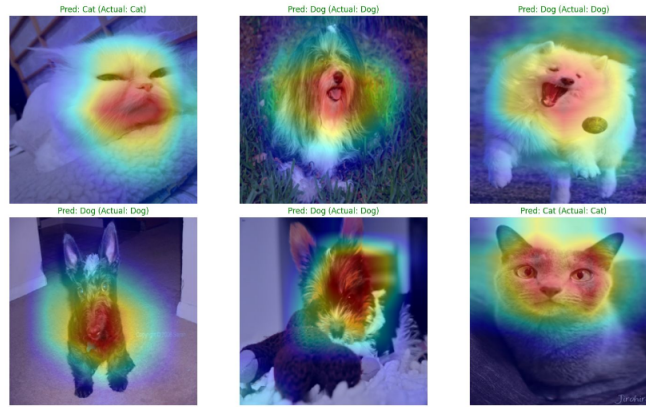


Figura 19: Grad-CAM: mapas de calor por parches.

7.6. Ataques Adversarios

Se busca el mínimo cambio en píxeles para que la red clasifique una imagen como otra clase. Se fija la nueva clase objetivo y se modifica la imagen para maximizar esa probabilidad.

7.7. Style Transfer

Transferir el estilo visual de una imagen a otra. Se optimiza una imagen intermedia I minimizando:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{content}} + \beta \cdot \mathcal{L}_{\text{style}}$$

Content Loss: diferencia entre las activaciones de I y la imagen de contenido en capas intermedias de una CNN (e.g. VGG). Preserva la estructura.

Style Loss: diferencia entre las **matrices de Gram** (correlaciones entre feature maps) de I y la imagen de estilo. Capturan texturas y patrones sin importar la posición espacial.

8. Transfer Learning

Transfer Learning

Reutilizar capas preentrenadas en un dataset grande (como ImageNet) para otro problema distinto.

- Entrenar desde cero es muy costoso.
- Las primeras capas capturan bordes, formas y texturas útiles para casi cualquier tarea visual.
- La última capa FC (específica del problema original) se reemplaza por una nueva.
- Se decide cuánto reentrenar: congelar todo (solo nueva capa) o fine-tuning (algunas capas previas).

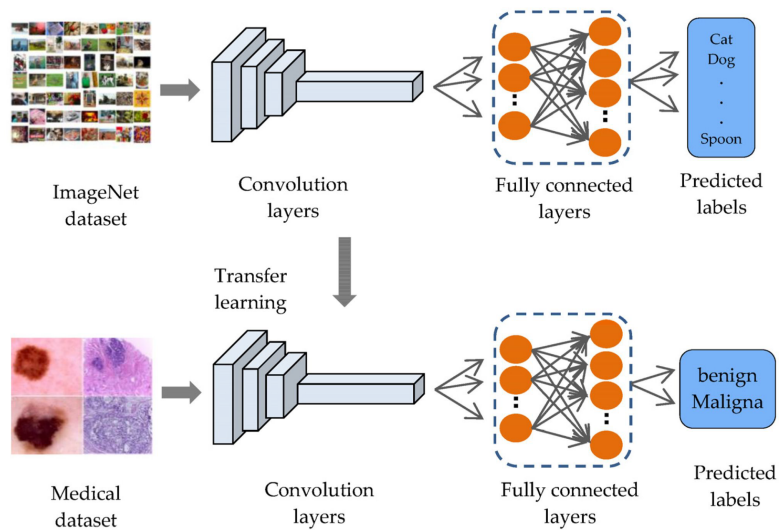


Figura 20: Transfer Learning: qué capas reutilizar y qué reemplazar.

9. Problemas sobre imágenes

9.1. Object Detection

No solo clasificar objetos, sino **localizarlos** con una bounding box.

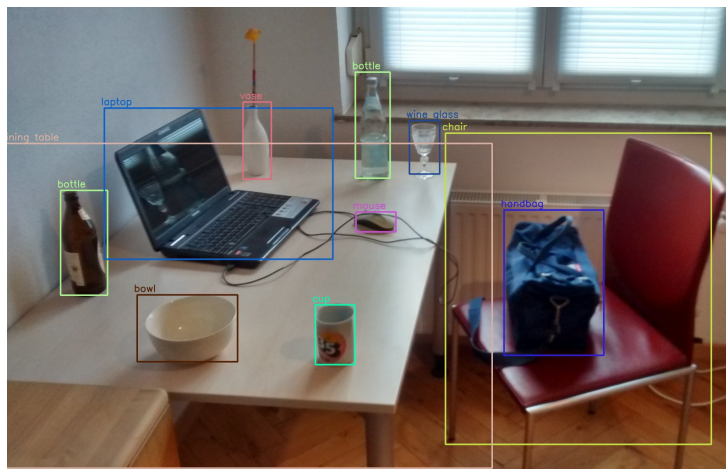


Figura 21: Object detection: clasificación + localización.

9.1.1. R-CNN

Usa **regiones candidatas** en vez de sliding window. Se hace inferencia sobre cada región.

1. Input image

2. Extract region proposals (~2k)

3. Compute CNN features

4. Classify regions

warped region

CNN

aeroplane? no.
:
person? yes.
:
tvmonitor? no.

9.1.2. Fast R-CNN

9.1.3. Faster R-CNN

9.1.4. YOLO

Non-Max Suppression: muchas celdas detectan el mismo objeto. Se toma la caja con mayor confianza y se descartan las que tengan alta intersección (IoU) con ella.

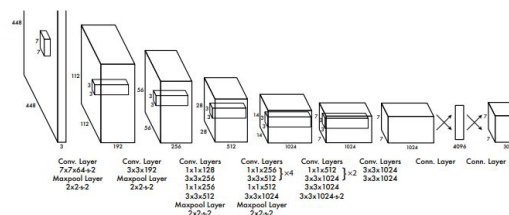


Figura 23: Arquitectura YOLO.

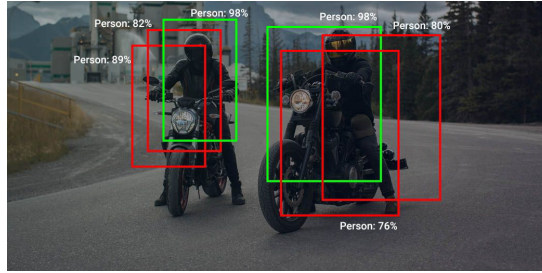


Figura 24: Non-Max Suppression.

9.2. Image to Image (I2I)

Una red puede tener tantas salidas como entradas: recibe una imagen y devuelve otra.

9.2.1. Autoencoders

Autoencoder

Red que reconstruye su propia entrada:

- **Encoder:** comprime la imagen hasta una **representación latente** (down-sampling).
- **Decoder:** expande la representación latente hasta reconstruir la imagen original.
- **Loss:** comparación píxel a píxel entre entrada y salida (no requiere etiquetas).

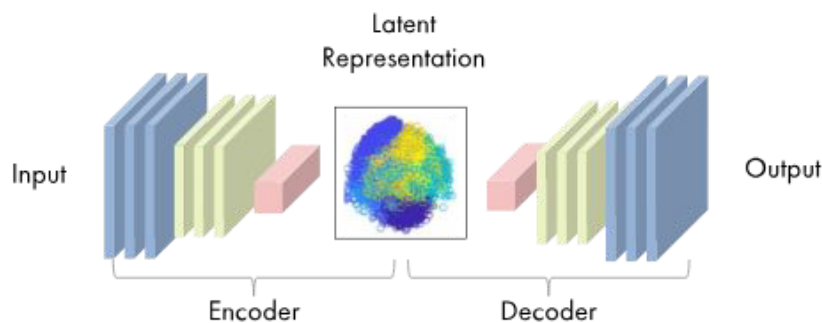


Figura 25: Arquitectura de un autoencoder.

Usos:

- **Reducción de dimensionalidad:** usar solo el encoder.
- **Image retrieval:** comparar representaciones latentes.
- **Denoising:** la red aprende a eliminar ruido.
- **Inpainting:** reconstruir regiones faltantes de la imagen.

9.2.2. Segmentación semántica

Clasificar cada píxel de la imagen indicando a qué clase pertenece.

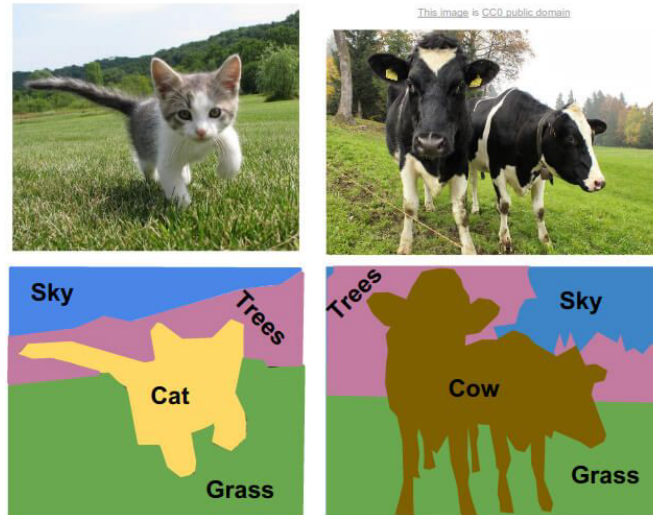


Figura 26: Segmentación semántica: cada píxel recibe una clase.

Loss functions posibles: Dice Loss, Jaccard Loss, Focal Loss. Se suele preferir **Dice Loss** sobre cross-entropy porque las clases están muy desbalanceadas (mucho fondo, poco objeto). Dice mide directamente la superposición:

$$\text{Dice} = \frac{2|X \cap Y|}{|X| + |Y|}$$

10. Generación de imágenes

10.1. GANs (Generative Adversarial Networks)

Dos redes entrenadas simultáneamente:

- **Generador:** crea imágenes a partir de ruido.
- **Discriminador:** determina si una imagen es real o generada.

10.2. Modelos de Difusión

Modelo	Idea
DDPM	Sumar ruido progresivamente a una imagen real hasta destruirla. Entrenar una U-Net para deshacer el ruido.
DDIM	Entrenamiento análogo pero con procedimiento más eficiente.
Stable Diffusion	Difusión en el espacio latente (no en la imagen directamente) para mayor velocidad y estabilidad.

10.3. CLIP

Conecta texto e imágenes: redes separadas llevan imágenes y texto a espacios latentes distintos, y luego se mapean para unirlos. Permite búsqueda cross-modal.

11. Secuencias, texto y NLP

11.1. Dificultades del texto

Los lenguajes varían en símbolos, orden y separación de palabras. Convertir texto a números no es trivial, a diferencia de las imágenes (que siempre son píxeles).

11.2. Representaciones de texto

11.2.1. Bag-of-Words

Se olvida del orden y crea un histograma de palabras.

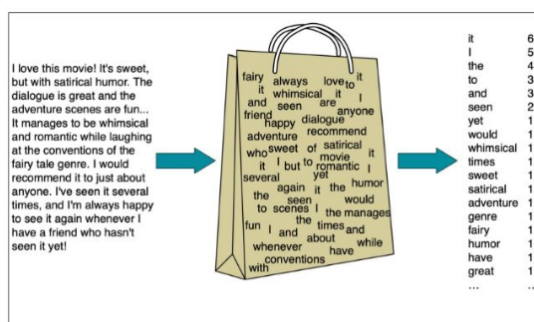


Figure 4.1 Intuition of the multinomial naive Bayes classifier applied to a movie review. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Figura 27: Representación Bag-of-Words.

Problemas: tantas columnas como palabras, pérdida de orden, ignora contexto y polisemia.

11.2.2. TF-IDF

Normalización que le quita peso a palabras muy frecuentes en el corpus. TF mide frecuencia en el documento; $IDF = \log(N/n_t)$ penaliza palabras que aparecen en muchos documentos. Producto $TF \times IDF$ da más peso a palabras discriminativas.

11.2.3. One-Hot Encoding

Cada token es su propia categoría. Sin semántica: el producto interno entre dos tokens es siempre cero.

11.2.4. Label Encoding

Cada token se traduce a un número. Representación simple pero sin semántica real.

11.3. Word2Vec

Word Embeddings

Llevar palabras a un espacio latente donde las relaciones semánticas se preservan como operaciones vectoriales. Se caracteriza una palabra a partir de las palabras que la rodean.

Dos modelos:

- **CBOW**: predecir la palabra del medio a partir de las n anteriores y n posteriores.
- **Skip-Gram**: proceso inverso, predecir las palabras de contexto a partir de la central.

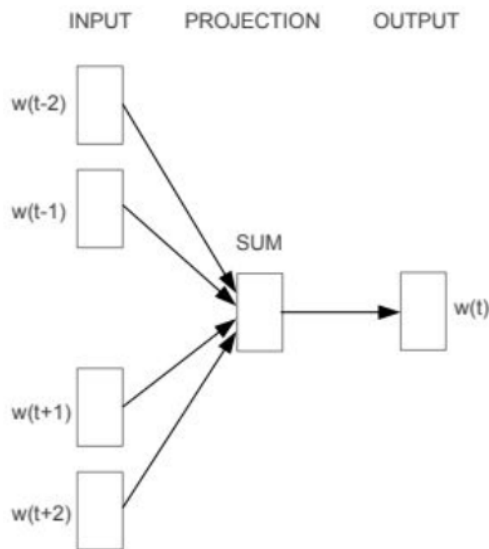


Figura 28: Arquitectura CBOW.

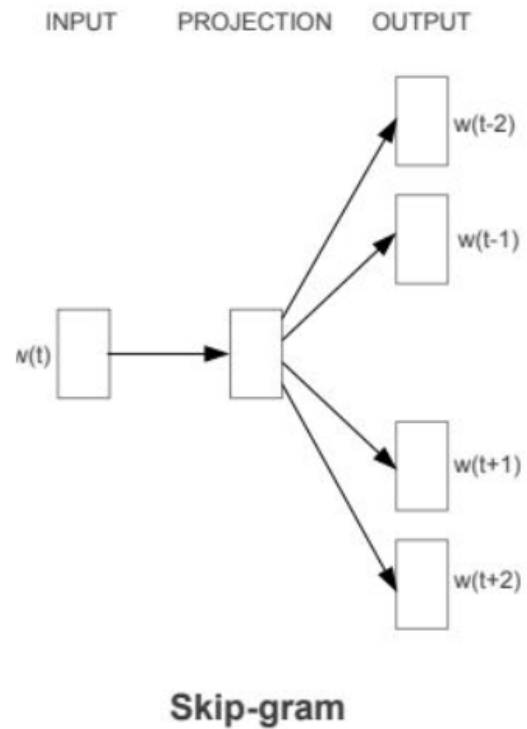


Figura 29: Arquitectura Skip-Gram.

Proceso CBOW:

1. Tomar palabras de contexto (anteriores y posteriores).
2. Codificar cada una en OHE.
3. Multiplicar por W para llevar a un espacio latente denso.
4. Promediar los vectores.
5. Proyectar con W' a logits de dimensión vocabulario.
6. Softmax $\Rightarrow$ probabilidades.
7. Comparar con ground truth y actualizar W y W' .

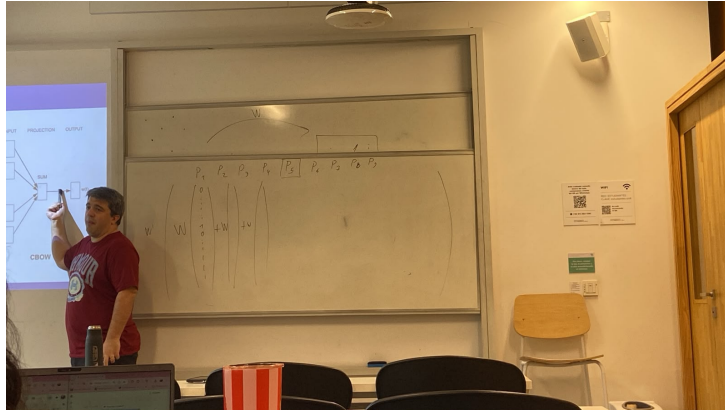


Figura 30: Pipeline completo de CBOW.

Analogías vectoriales: $v_{\text{madrid}} \approx v_{\text{paris}} - v_{\text{francia}} + v_{\text{españa}}$. Se busca la palabra cuyo vector sea más cercano al resultado.

Product2Vec: misma idea aplicada a secuencias de productos para encontrar productos similares.

11.4. Language Models

Buscan predecir la siguiente palabra en una secuencia. Se reduce a un problema de clasificación sobre todo el vocabulario.

12. Redes Neuronales Recurrentes (RNNs)

RNN

Permiten procesar secuencias de tamaño variable (a diferencia de las redes feedforward con inputs fijos). El **bloque recurrente** recibe el input x_t y una retroalimentación del resultado anterior:

$$h_t = \sigma(W_1 x_t + W_2 h_{t-1})$$

El hidden state h_t codifica información de todas las palabras procesadas anteriormente. La retroalimentación se corta en algún punto.

Backpropagation Through Time (BPTT): al desenrollar la RNN, los gradientes dependen de la exponenciación de la matriz de pesos W_{hh} . Multiplicar muchas veces la misma matriz hace que los gradientes se desvanezcan (o exploten). En la práctica, secuencias de 25–30 pasos ya sufren este problema $\Rightarrow$ las RNNs no pueden aprender dependencias lejanas.

13. LSTM, Seq2Seq y Attention

13.1. LSTM (Long Short-Term Memory)

LSTM

Tipo de RNN que soluciona el problema de que el gradiente se desvanece para aprender dependencias lejanas.

Dos vectores de estado:

- **Hidden state** $\vec{h}$: memoria a corto plazo del contexto actual.
- **Cell state** $\vec{c}$: memoria a largo plazo que pasa información sin desvanecerse.

Gates (vectores que controlan el flujo de información):

- **Forget Gate**: decide qué *borrar* del cell state. Sigmoide: 0 = olvidar, 1 = recordar.
- **Input Gate**: decide *cuánto* de lo nuevo se escribe en c (sigmoide) y *qué* (tanh).
- **Output Gate**: decide cuánto del cell state se expone como salida h_t .

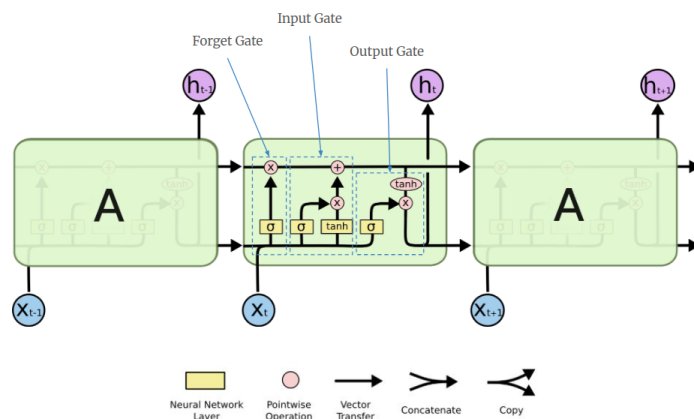


Figura 31: Arquitectura interna de una celda LSTM.

El “+” en el cómputo de C_t permite que la información fluya sin desvanecerse (análogo a las skip connections de ResNet). Si forget gate = 1 e input gate = 0, el cell state fluye intacto.

13.2. BiLSTM

Misma idea que LSTM pero en **ambas direcciones**: izquierda a derecha y derecha a izquierda, obteniendo el doble de hidden states. Captura contexto de ambos lados de cada token.

13.3. GRU (Gated Recurrent Unit)

Versión simplificada de LSTM: combina c y h en un solo vector, y fusiona forget e input gates. Menos parámetros, rendimiento similar.

13.4. Limitaciones de LSTM/BiLSTM

- Procesan secuencialmente $\Rightarrow$ **no paralelizables**.
- Memoria a largo plazo limitada (mejor que RNN, pero no infinita).
- Resumen todo en un vector fijo final.

13.5. Seq2Seq

Seq2Seq

Toma secuencias para generar otras secuencias (e.g. traducción). Usa dos LSTMs:

- **Encoder**: procesa la secuencia de entrada y genera una representación.
- **Decoder**: genera la secuencia de salida a partir de esa representación.

En entrenamiento se usa **teacher forcing**: en cada paso del decoder se ingresa la palabra correcta aunque no haya sido la predicha.

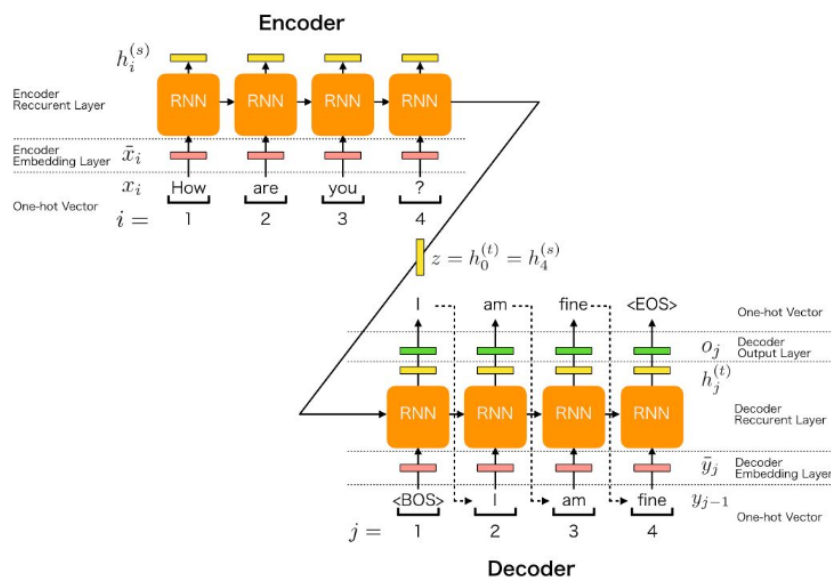


Figura 32: Arquitectura Seq2Seq con encoder-decoder.

13.6. Attention

Mecanismo que permite al decoder “mirar” todas las posiciones de la secuencia de entrada (no solo el último hidden state del encoder), asignando pesos de importancia a cada posición según su relevancia para la palabra que se está generando.

Supongamos que estamos generando la tercera palabra de la traducción. Ya tenemos todos los hidden states h_i del encoder y el estado s_{t-1} del decoder. Se calculan **scores de atención** para cada h_i :

$$e_{t,i} = v_a^\top \tanh(W_a s_{t-1} + U_a h_i)$$

donde v_a, W_a, U_a son pesos aprendibles. Luego se pasan por softmax para obtener pesos

$\alpha_{t,i}$ y se calcula el **vector de contexto**:

$$c_t = \sum_i \alpha_{t,i} h_i$$

Este vector c_t se concatena con la entrada del decoder en el paso t . La semántica es: “para dar la siguiente palabra, enfocate en esta parte de la frase original”.

Cuello de botella Seq2Seq: sin Attention, toda la información de la frase de entrada se comprime en un único vector h final del encoder. Funciona para frases cortas, pero no para párrafos. Attention resuelve esto dejando al decoder ver todos los h_i intermedios.

14. Transformers

14.1. Problema de secuencialidad de las RNNs

Aunque Attention resolvió el cuello de botella en la representación, las RNNs siguen siendo **inherentemente secuenciales**: para conocer el estado en el paso 50, primero hay que computar los 49 anteriores. No son paralelizables. Las CNNs no tenían este problema y por eso entre 2012–2016 aprovecharon mejor los avances en hardware.

14.2. Attention Is All You Need (2017)

El paper de Vaswani et al. propuso el **Transformer**: no es necesario usar RNNs. El mismo mecanismo de atención que relacionaba encoder con decoder se puede usar como ladrillo fundamental.

14.3. Self-Attention

En la atención clásica (Bahdanau), la atención era **cruzada** entre la frase del decoder y la del encoder. En **Self-Attention**, las palabras de una misma oración se prestan atención a sí mismas para construir una representación rica.

14.4. Scaled Dot-Product Attention

Scaled Dot-Product Attention

Paso a paso:

1. Secuencia de palabras w_i .
2. Convertir a embeddings: $x_i = E w_i$.
3. Transformar cada embedding en su **Query** $q_i = Qx_i$, **Key** $k_i = Kx_i$ y **Value** $v_i = Vx_i$.
4. Calcular similaridades: $e_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$.
5. Normalizar con softmax: $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}$.
6. Salida: suma ponderada de values: $o_i = \sum_j \alpha_{ij} v_j$.

En forma matricial:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Intuición: es como una *lookup table suavizada*. En un diccionario, se tiene una query q , se busca la key k exacta y se toma su value v . En self-attention, la query tiene correspondencia parcial con varias keys y se toma el promedio ponderado de los values.

14.5. Preguntas Clave del Mecanismo

- **¿De dónde salen Q, K, V?** Son matrices de pesos aprendibles. Se inicializan random y se ajustan con backpropagation.
- **¿Por qué dividir por $\sqrt{d_k}$?** Cuando la dimensión es grande, el producto interno crece mucho. La división estabiliza las cuentas (de ahí “scaled”).
- **¿Por qué aprender Q y K por separado?** La multiplicación $q_i^\top k_j$ es una **low-rank factorization** de la matriz de scores e_{ij} . Más eficiente computacionalmente y obliga a un cuello de botella en la representación.
- **¿Qué rompe la linealidad?** La softmax rompe un poco, pero el objetivo principal de no linealidad viene de las capas feed-forward con funciones de activación que se agregan después.

14.6. Masked Attention

En el decoder, las palabras se generan de forma online: al tener la palabra t , no se conoce la $t + 1$. Se **enmascaran** los scores de palabras futuras fijándolos en $-\infty$ para que la softmax les asigne peso cero.

14.7. Positional Encoding

Positional Encoding

Sin recurrencia, self-attention procesa todas las palabras en paralelo y pierde la noción de orden (se vuelve un bag-of-words). La solución: **sumar un vector de posición** a cada embedding.

El paper original usó funciones de senos y cosenos de distintas frecuencias:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Evolución posterior: Positional Encoding aprendible $\rightarrow$ Relative PE $\rightarrow$ Rotary PE (RoPE).

14.8. Multi-Head Attention

Multi-Head Attention

Análogo a tener múltiples filtros en una CNN: sobre las mismas palabras se quieren aprender **diferentes representaciones**. Una cabeza puede aprender a reconocer entidades (países, personas), otra a relacionar verbo y sujeto.

Se ejecutan h mecanismos de atención en paralelo, cada uno con sus propias matrices Q_i, K_i, V_i . Las salidas se concatenan y se proyectan con una capa lineal.

Son computacionalmente muy eficientes.

14.9. Arquitectura del Transformer

Sigue teniendo **encoder** y **decoder**, pero sin RNNs:

- El encoder tiene capas de **self-attention** + feed-forward.
- El decoder tiene capas de **masked self-attention** + **cross-attention** (con el encoder) + feed-forward.
- **Add & Norm**: después de cada bloque hay una conexión residual (como ResNet) y Layer Normalization (reemplazo de BatchNorm). Ayudan al flujo de gradientes y aceleran convergencia.

Ya en el trabajo original se consiguieron resultados estado del arte en traducción automática con mucho menos entrenamiento que los modelos previos (LSTMs).

14.10. BERT vs GPT

	BERT (2018, Google)	GPT (2018, OpenAI)
Parte usada	Solo encoder	Solo decoder
Dirección	Bidireccional	Autorregresivo (izquierda a derecha)
Uso	Clasificación, NER, Sentiment Analysis (requiere fine-tuning)	Generación de texto
Pre-entrenamiento	Masked Language Model (predecir palabras ocultas) + Next Sentence Prediction	Predecir la siguiente palabra

14.11. Evolución de GPT

- **GPT-1 (2018)**: El foco era resolver problemas de NLP con fine-tuning. Perdía contra BERT.
- **GPT-2 (2019)**: Foco en resolver tareas NLP sin fine-tuning (**zero-shot learning**).
- **GPT-3 (2020)**: Foco completamente en el Language Model y todo lo que puede resolver sin ajuste adicional.

14.12. InstructGPT y alineación

InstructGPT y RLHF

Un Language Model bueno genera secuencias gramaticalmente correctas, pero puede no estar **alineado** con la intención del usuario. Ejemplo: ante una pregunta, GPT-3 completaba con opciones multiple choice como si fuera un examen.

InstructGPT resolvió esto con **Reinforcement Learning from Human Feedback (RLHF)**:

1. Se entrena un modelo de recompensa con preferencias humanas.
2. Se usa RL para ajustar el LLM y que maximice esa recompensa.

ChatGPT (nov. 2022) usó GPT-3 como backbone + técnicas de alineación para ser conversacional.

14.13. LLMs: Más que un predictor de palabras

El backbone de un LLM sigue siendo el decoder de un Transformer, pero hay componentes adicionales:

- **Tokenizers**: cómo se segmenta el texto en tokens.
- **Data**: calidad y cantidad de datos de entrenamiento.
- **SFT y RLHF**: Supervised Fine-Tuning y alineación con feedback humano.
- Optimizaciones de cómputo.

14.14. Vision Transformers (ViT)

ViT (2021)

Idea análoga a “Attention Is All You Need” pero para imágenes: no hacen falta CNNs. Se parte la imagen en **parches** y se aplican mecanismos de atención entre ellos.

Buenos resultados pero con tiempos de cómputo altos (prohibitivos para resoluciones altas). La batalla CNN vs ViT sigue abierta para visión, a diferencia de texto donde el Transformer ganó claramente.

14.15. Tipos de Transformers (resumen)

Tipo	Ejemplo	Uso
Encoder-Only	BERT	Comprender texto: clasificación, sentiment, NER
Encoder-Decoder	T5, Transformer	Seq2Seq: traducción, summarization
Decoder-Only	GPT	Generar texto/secuencias. Predecir próximo token

14.16. Decoder-Only Transformers

Entrenados bajo la tarea de predecir el próximo token. Cada token solo puede mirar tokens anteriores (**causal masking**). Tres componentes:

1. **Embedding**: tokenización + positional encoding.
2. **Bloque Transformer**: Masked Multi-Head Attention + Feed-Forward (MLP).
3. **Output**: proyección lineal + softmax sobre todo el vocabulario.